Sisteme Distribuite

Laboratorul 1.

Introducere în limbajul de programare Go

Andrei Pătrașcu

Răzvan Anghel

Introducere

Ce este limbajul de programare Go?

Go, numit și Golang, este un limbaj de programare open-source lansat de Google în 2012. Scopurile sale principale au fost simplitatea și viteza limbajului, respectiv exploatarea eficientă a procesoarelor multicore, care au început să apară în volume mari pe piață la sfârșitul anilor 2000. Caracteristici importante:

- limbaj de programare high-level
- sistem strict de tipuri (nu se pot efectua operații asupra variabilelor de tipuri diferite)
- type-checking static (nu se pot schimba tipurile după declarare decât prin conversie)
- limbaj compilat
- garbage collected
- procese care comunică prin canale
- sintaxă similară cu C++
- o bibliotecă standard cu multe funcționalități
- folosește implicit UTF8

Go este încă un limbaj de programare relevant, fiind folosit în sisteme precum Docker, Kubernetes, Google Cloud, PayPal și infrastructuri backend (Cloud, DevOps) [1, 4]. Practic, orice sistem care presupune scalabilitate poate fi construit în Go. De asemenea, el devine un limbaj popular și pentru backend-ul aplicațiilor web.

Instalare și concepte de bază

Instalare

În cadrul acestui laborator vom folosi cea mai recentă versiune a limbajului Go (*1.27.1* la momentul scrierii acestui document). Nu ar trebui să apară probleme de compatibilitate cu alte versiuni, dar vă sfătuim să folosiți mereu versiunea cea mai recentă și să faceți update-uri limbajului, atunci când acestea sunt disponibile. Pentru instrucțiuni detaliate de instalare pe fiecare sistem de operare, urmați pașii descriși în [2].

Pentru a verifica că totul este în regulă, porniți un terminal nou și rulați comanda `go version`. Ar trebui ca output-ul să arate cam așa:

```
go version go1.27.1 windows/amd64
go version go1.27.1-X:nodwarf5 linux/amd64
```

Creearea unui proiect (pachete și module)

Fișierele `.go` vor conține codul care va fi compilat pentru a forma executabilul final. Ele se organizează în pachete și module. Similar cu limbajul Java, un **pachet** în Go reprezintă un director care conține mai multe fișiere `.go` și care vor fi compilate împreună. Într-un pachet poate fi prezentă cel mult o singură funcție `main()` și nu pot exista mai multe funcții cu același nume, cu excepția cazului când ele sunt metode pentru tipuri diferite. Un pachet nu va genera un executabil decât dacă are inclusă o funcție `main()` [3, 5].

Un modul, pe de altă parte, este format din mai multe pachete, fiind forma de organizare preferată în cazul unui proiect de dimensiuni mai mari. Modulele sunt definite de fișiere `go.mod` care conțin: numele modulului, versiunea Go folosită și dependințele folosite în modul. De asemenea, folosirea modulelor oferă mecanisme de versionare a proiectului și un namespace pentru a fi folosit la importuri în alte module [3, 5]. Pentru a crea un proiect nou, mergeți într-un folder gol și rulați comanda: `go mod init laborator_1`. În interiorul folderului, creați un fișier `main.go` cu următorul conținut:

Structura unui fișier .go

```go
// Primul lucru care se precizeaza este package-ul din care fisierul face parte
package main

// A doua etapa este importarea pachetelor (nefolosirea unui pachet genereaza o eroare)
import "fmt"

// A treia etapa este scrierea functiilor si a tipurilor. Aici avem doar o functie main:
func main() {

    fmt.Println("Hello, world!")
}
```

Câteva comenzi utile:

Comandă	**Descriere**
`go version`	Afișează versiunea curentă de Go.
`go mod init nume_modul`	Creează un modul Go nou cu numele specificat. De regulă, numele este direct un repository de pe GitHub, dar nu este obligatoriu.
`go build file.go`	Compilează fișierul. Dacă acesta face parte din pachetul `main` și există o funcție **main()** în fișier, se generează și un executabil.
`go build .`	Compilează directorul curent. Dacă acesta conține un pachet `main`, generează și un fișier executabil. Din cauza setărilor default ale limbajului, folosim această comandă doar când avem și un modul inițializat.
`go run file.go`	Compilează și rulează fișierul `file.go`. Dacă fișierul face parte din pachetul `main` și conține și o funcție `main()` se va crea și un executabil temporar, care va fi șters imediat după rulare. Această comandă este utilă dacă avem mai multe fișiere independente (de exemplu exerciții) într-un folder, pe care vrem să le rulăm separat.
`go run .`	Compilează și rulează folder-ul curent, dacă acesta conține pachetul `main`, altfel va arunca o eroare. Și aici preferăm utilizarea comenzii doar dacă există deja un modul inițializat.

Practic, pentru a genera un executabil dintr-un fișier `.go`, este necesar ca el să facă parte din pachetul `main` și să conțină o funcție `main()`. Altfel, el poate fi doar compilat.

Comentarii și operatorul punct și virgulă(;)

Comentariile sunt delimitate de caracterele slash și steluță (//, respectiv /* și */ pentru comentarii pe mai multe rânduri).

În Go se pot folosi punctul și virgula (;) la finalul unui rând, dar nu este necesar, nici recomandat, fiind contrat viziunii de simplitate a limbajului.

Tipuri de date

Tabelul de mai jos prezintă principalele tipuri de date din limbajul Go și caracteristicile acestora:

Tip de date	**Descriere**	**Detalii / Constrângeri**
`int` (*implicit 32/64*), `int8`, `int16`, `int32`, `int64`, `uint` `byte`, `rune`	Tipuri numerice întregi.	Valori întregi cu și fără semn. Dacă nu există constrângeri de semn sau mărime, ar trebui folosit `int`.
`float32`, `float64`	Tipuri numerice cu virgulă mobilă.	**Nu** există un tip `float` simplu, nici `double`.
`string`	Șir de caractere delimitat prin ghilimele (`"`).	**Nu** poate stoca mai multe linii.
`bool`	Valori booleene.	Poate fi doar `true` sau `false`, mereu cu litere mici.

Declarare. Constante și variabile

Pentru orice tip de date se pot declara atât constante cât și variabile, folosind cuvintele cheie `var`, respectiv `const`. O variabilă sau o constantă nefolosită va declanșa o eroare.

```
// Sintaxa generala
tip_mutabilitate nume_variabila tip_variabila

// Exemple
var integer_number int
var float_number float64
const name string = "Sisteme Distribuite"
```

Variabilele neinițializate, sunt inițializate automat cu o **valoare default de tip** (0, "", false, nil). Tipul poate fi **dedus automat** de Go, folosind operatorul **:=**, după cum urmează:

```
myVar := "text" // echivalent cu var myVar string = "text"
var1, var2 := 1, 2  // variabilele de acelasi tip se pot declara si pe
var var3, var4 int = 3, 4  // aceeasi linie, separate prin virgula
```

Constantele nu pot fi neinițializate, deoarece valoarea lor trebuie să fie deja cunoscută la compilare. De asemenea, operatorul **:=** nu poate fi folosi pentru a inițializa constante. În schimb, deducerea automată a tipului se poate face automat și aici prin nespecificarea acestuia:

```
const Pi = 3.14159
const WelcomeMessage = "Hello, World!"
```

Operatori

Operatorii care se pot aplica numerelor sunt: **+**, **-**, ***, **/**, **%**. Ele se pot folosi doar între același tip de date. Dacă vrem să efectuăm operații între tipuri diferite, atunci sunt necesare conversii:

```go
var floatNum float32 = 7.2
var intNum = 4
var result float32 = floatNum + intNum // => eroare

var result float32 = floatNum + float32(intNum) // => corect
```

Împărțirea între întregi are ca rezultat un număr întreg și e aproximată prin lipsă. Împărțirea între un întreg și un `float` are ca rezultat un `float`.

Specificatori de formatare

Specificator	Descriere / Utilizare
`%v`	Valoarea într-un format implicit, foarte ușor de citit.
`%+v`	Folosit pentru structuri: include și numele câmpurilor.
`%T`	Afișează doar tipul de dată.
`%t`	Valoare booleene.
`%b`	Număr întreg reprezentat în baza 2.
`%d`	Număr întreg reprezentat în baza 10.
`%x` / `%X`	Baza 16 cu litere mici, respectiv mari.
`%f`	Număr cu virgulă mobilă (*floating-point*).
`%s`	Șir de caractere (*string*).

Funcții pentru afișare

Funcție	Descriere	Exemplu
`fmt.Print()`	Concatenează argumentele fără linie nouă la final. Adaugă spațiu între operanzi doar dacă niciunul nu este șir.	`fmt.Print("Salut", 42)` → `Salut42`
`fmt.Println()`	Afișează argumentele adăugând mereu spațiu între ele și o linie nouă (`\n`) la final.	`fmt.Println("ID:", 10)` → `ID: 10\n`
`fmt.Printf()`	Formatează datele pe baza unui șablon (*format string*) și le afișează.	`fmt.Printf("%s are %d ani", "Dan", 20)`
`fmt.Sprintf()`	Formatează argumentele în funcție de șablon (*format string*) și returnează rezultatul ca `string`.	`msg := fmt.Sprintf("%s:%d", "Port", 8080)`

Instrucțiuni de Control în Go

Instrucțiunea `if`

Sintaxa de bază pentru instrucțiunea `if` este:

```go
if conditie {
    // bloc de cod
} else if conditie {
    // alt bloc de cod
} else {  // se executa cand toate celelalte sunt false
    // alt bloc de cod
}
```

- Instrucțiunea `if` nu conține paranteze, nici măcar atunci când se leagă mai multe condiții cu operatori logici.
- Evaluarea condițiilor este scurtcircuitată!
- Nu este nevoie de paranteze la condiții, dar acoladele sunt obligatorii, chiar dacă blocul de cod are o singură linie!

Instrucțiunea `switch`

```
switch expresie {
case conditie1, conditie2: // ele vor fi tratate ca operator ||
    // acoladele nu sunt necesare
default:  // aceasta ramura este optionala
    // ...
}
```

- Break-ul este scris de compilator în fiecare ramură!
- Un switch fără expresie este echivalent cu mai multe instrucțiuni `if-else`:

```
t := time.Now()
switch {
case t.Hour() < 12:
    fmt.Println("Este dimineata")
default:
    fmt.Println("Este dupa amiaza")
}
```

Instrucțiunea `for`

Limbajul Go nu include o instrucțiune `while` dedicată, aceasta fiind înlocuită de instrucțiunea `for`. Ea apare în mai multe forme:

1. For cu o singură condiție (echivalent cu while):

```
i := 0
for i <= 2 {
    // bloc de cod; de exemplu:
    fmt.Println(i)
    i += 1   // Aici este necesara incrementarea manuala a lui i pentru a
evita o bucla infinita
}
```

2. For clasic, cu inițializare și 2 condiții:

```
for j := 0; j < 3; j++ {   // Se pot folosi si alte combinatii de operatori
   // bloc de cod
}
```

3. For cu range (exclusiv):

```
for i := range 3 {  // Aici, i este incrementat automat de la 0 la 2
    // bloc de cod
}
```

4. For fără condiție (loop infinit):

```go
for { // Loop infinit pana la break sau return
    // bloc de cod
}
```

Operații I/O

Citirea linie cu linie dintr-un fișier sau de la standard input se realizează cu un obiect de tip `bufio.Scanner` din pachetul `bufio`. Pentru a gestiona erori în Go, o funcție care poate arunca o excepție, trebuie să returneze obligatoriu și un obiect de tip `error`. Acest design pattern conduce la o verificare mult mai simplă și imediată a potențialelor erori, după cum se poate vedea mai jos în exemplul de citire:

```go
// Deschidem fisierului si gestionam potentialele erori
file, err := os.Open("file.txt")
if err != nil {
    fmt.Println(err)
}

// Închide fisierul imediat ce se întampina o eroare sau la finalul citirii
// Mai multe informatii în laboratorul 2
defer file.Close()

// Creem un nou Scanner
scanner := bufio.NewScanner(file)

// Sau folosim bufio.NewScanner(os.Stdin) pentru standard input.
// În acest caz, nu mai avem nevoie nici de open nici de close.

// Citim linie cu linie prin metoda Scan()
for scanner.Scan() {
   fmt.Println(scanner.Text()) // returneaza linia citita ca string
}

// De fiecare data, verificam daca citirea s-a finalizat cu succes
err = scanner.Err()
if err != nil {
    fmt.Println("Eroare la citire: ", err)
}
```

Structuri de date

Array-uri (Tablouri cu dimensiune statică)

În Go, un **array** este o secvență continuă de memorie cu o lungime fixă, care stocheaează elemente de același tip. Ele sunt indexate de la 0, iar toate elementele sunt inițializate cu valorile nule de tip. În declarare se specifică și dimensiunea:

```go
// Declarare simpla (elementele sunt initializate cu 0):
var intArr [3]int32

// Declarare cu initializare:
var intArr [3]int32 = [3]int32{1, 2, 3}
intArr := [3]int32{1, 2, 3}
```

```go
// Declarare cu deducerea automata a numarului de elemente
intArr := [...]int32{1, 2, 3}

// Tiparire directa
fmt.Println(intArr) // Va afisa [1 2 3]

// Pentru a determina lungimea unui array, folosim functia len:
fmt.Println(len(intArr))
```

Slice-uri (tablouri cu dimensiune dinamică)

Un **slice** este o structură de date dinamică construită ca un wrapper peste un array. Astfel, toate elementele sunt de același tip, iar mărimea lui se poate schimba. Funcționează asemănător clasei Vector din C++ sau clasei ArrayList din Java. Tot similar cu aceste structuri, slice-urile au o lungime, care exprimă numărul curent de elemente și o capacitate care exprimă numărul maxim de elemente care se pot insera până la o nouă realocare. Elementele individuale din slice-uri se accesează identic cu cele din array-uri.

Un slice se poate declara în două moduri:

- direct: `intSlice := []int32{1, 2, 3}`
- prin funcția `make([]tipul_de_date_al_sliceului, len, cap)`: stabilește lungimea și capacitatea inițială.

Un element poate fi inserat într-un slice, folosind funcția `append(slice, element)`. La fel ca în Python, putem folosi operatorul `[low:high]` pentru a extrage elementele din slice cuprinse între indicele `low` și `high - 1`. Exemple cu operații pe slice-uri:

```go
// Initializare
intSlice := []int32{1, 2, 3}
fmt.Println(intSlice[2])  // Elementele se pot accesa ca la array-uri

intSlice = append(intSlice, 7)  // Se poate face append cu un singur element
intSlice = append(intSlice, 10, 14, 12)  // Sau cu mai multe

// Nu se poate face append un slice la alt slice
intSlice2 := []int32{8, 9}
intSlice = append(intSlice, intSlice2)  // => eroare

// În schimb, se poate folosi operatorul de despachetare ...
intSlice = append(intSlice, intSlice2...)

// Alocare folosind functia make(tip, lungime_initiala, capacitate_initiala):
intSlice3 := make([]int32, 2, 5)

// Se extrag elementele cu indicii 1 si 2
slice_mai_mic := intSlice[1:3]  // va contine [2 3]
```

Map-uri (Tabele de dispersie sau dicționare)

Un **map** este o structură de date care stochează perechi de tip cheie-valoare și funcționează ca un dicționar din Python sau ca un tabel de dispersie.

Un map se poate crea folosind tot funcția `make`, cu parametrul `map[Tip_Cheie]Tip_Valoare`. Accesarea valorii asociate unei chei inexistente va returna valoarea nula de tip a valorii. Un map în Go poate returna și o valoare *bool* opțională, care va fi *true* dacă cheia se află în map și *false* dacă nu:

```go
// Initializare prin functia make:
var myMap map[string]uint8 = make(map[string]uint8)
```

```go
myMap2 := make(map[string]uint8)

// Verificarea existentei unei chei:
age, ok := myMap2["John"]
if !ok {
    // Cheia nu se afla în map
}

// Cu wildcard, putem verifica prezenta ignorand valoarea
_, ok := myMap2["Kevin"]
```

Pentru a goli complet un map, se folosește funcția `clear(mapName)`. Pentru a șterge o pereche cheie-valoare dintr-un map, se folosește funcția `delete(mapName, keyName)`. Ea nu returnează nicio valoare.

```go
// Stergerea unei perechi
delete(myMap2, "John")

// Golirea completa a dictionarului
clear(myMap2)
```

Important: la iterarea printr-un map, ordinea nu este mereu aceeași.

Iterarea colecțiilor

Keyword-ul `range` permite parcurgerea oricărei structuri de date din Go. El returnează diferite date, în funcție de colecția pe care o iterează și de numărul de variabile asupra căruia acționează:

- **Array-uri și Slice-uri:** Returnează perechea `(index, valoare)`. Dacă se folosește doar o variabilă, returnează indexul.
- **Map-uri:** Returnează perechea `(cheie, valoare)`. Dacă se folosește o singură variabilă, returnează cheia.

Exemple de iterații:

```go
// Range pe array/slice, doar cu index
for i := range intArr {
    fmt.Println(intArr[i])
}

// Range pe array/slice, cu index si valoare
for i, val := range intSlice {
    fmt.Println(i, val)
}

// Range pe array/slice, cu ignorarea indexului (folosind wildcard):
var sum int32
for _, num := range intSlice {
    sum += num
}

// Range pe map, cu cheie si valoare:
for name, age := range myMap {
    fmt.Printf("Name: %v, Age: %v\n", name, age)
}

// Si la map se pot ignora cheile, folosind wildcard:
for _, age := range myMap {
    fmt.Printf("Age: %v\n", age)
}
```

```go
// Range pe map, doar cu cheie:
for name := range myMap {
    fmt.Println(myMap[name])
}
```

Exerciții

1. Scrieți un program Go care citește un număr natural de la tastatură și verifică dacă este prim sau nu. **(1p)**

2. Scrieți un program Go care stochează un array în memorie. Programul trebuie să afișeze tabloul sortat crescător folosind o metodă de sortare la alegere. **(1p)**

3. Scrieți un program Go care stochează în memorie un șir de numere și care afișează un slice cu numerele pare din acel șir. **(1p)**

4. Scrieți un program care stochează în memorie un slice de `float32` și citește de la tastatură un index. Programul trebuie să elimine elementul de la poziția *index*, păstrând ordinea celorlalte elemente. **(1p)**

5. Scrieți un program Go care ia elementele unui slice de string-uri și îl modifică astfel încât elementele să apară în ordine inversă. Nu folosiți un alt slice pentru această operațiune. **(1p)**

   Exemplu: `slice := []string"Laboratorul", "de", "sisteme", "distribuite".`
   Rezultatul va fi: `["distribuite" "sisteme" "de" "Laboratorul"]`

6. Fie un map de tipul *string-int* care stochează perechi de tipul *nume-înălțime*. Să se afișeze numele persoanelor în ordinea descrescătoare a înălțimilor. Dacă două persoane au aceeași înălțime, nu contează ordinea de afișare. **(1p)**

   Exemplu de input:

   ```go
   map[string]int{
       "Andrei":  190,
       "Maria":   165,
       "Ion":     182,
       "Elena":   172,
       "George":  178,
       "Ana":     165,
   }
   ```

7. Scrieți un program Go care citește un slice de string-uri formate numai din litere mici dintr-un fișier. Programul trebuie să afișeze de câte ori apare fiecare cuvânt în slice-ul de string-uri. **(1p)**

   Exemplu de input: mar para mar strugure banana para mar strugure portocala

8. Angajații unei firme sunt stocați într-un dicționar `map[string]int` cu perechi de forma nume-id_departament. Creați un nou `map[int][]string` care să stocheze perechi de tipul id_departament-slice cu numele angajaților din departament. **(1p)**

   Exemplu:

   ```go
   map[string]int{
       "Ana Popescu":       1,
       "Mihai Ionescu":     2,
       "Bogdan Radu":       1,
       "Elena Dumitrescu": 3,
       "Cristian Vasile":   2,
       "Ioana Enache":      1,
   ```

```
}
```

Rezultatul va fi:
Departamentul 1: `[Ana Popescu, Bogdan Radu, Ioana Enache]`
Departamentul 2: `[Mihai Ionescu, Cristian Vasile]`
Departamentul 3: `[Elena Dumitrescu]`

9. Scrieți un program Go care stochează în memorie un array cu numere naturale. Programul va citi de la tastatură un număr `s` și va afișa în $O(n)$ indicii elementelor care însumate au ca rezultat `s`. **(2p)**

   Exemplu:
   2 7 11 8 4 9
   s = 15

   Rezultatul va fi:
   1 3
   2 4

Bibliografie

[1] Autori Wikipedia. Titlul articolului — Go (programming language). https://en.wikipedia.org/wiki/Go_(programming_language), 2026. [Ultima accesare: August 2026].

[2] Google și comunitatea Go. Download and Install. https://go.dev/doc/install, 2026. [Ultima accesare: Septembrie 2026].

[3] Google și comunitatea Go. Go Documentation. https://go.dev/doc/, 2026. [Ultima accesare: Septembrie 2026].

[4] Google și comunitatea Go. Go Use Cases. https://go.dev/solutions/use-cases, 2026. [Ultima accesare: August 2026].

[5] Mark McGranaghan and Eli Bendersky. Go by Example. https://gobyexample.com/, 2026. Ultima accesare: August 2026.